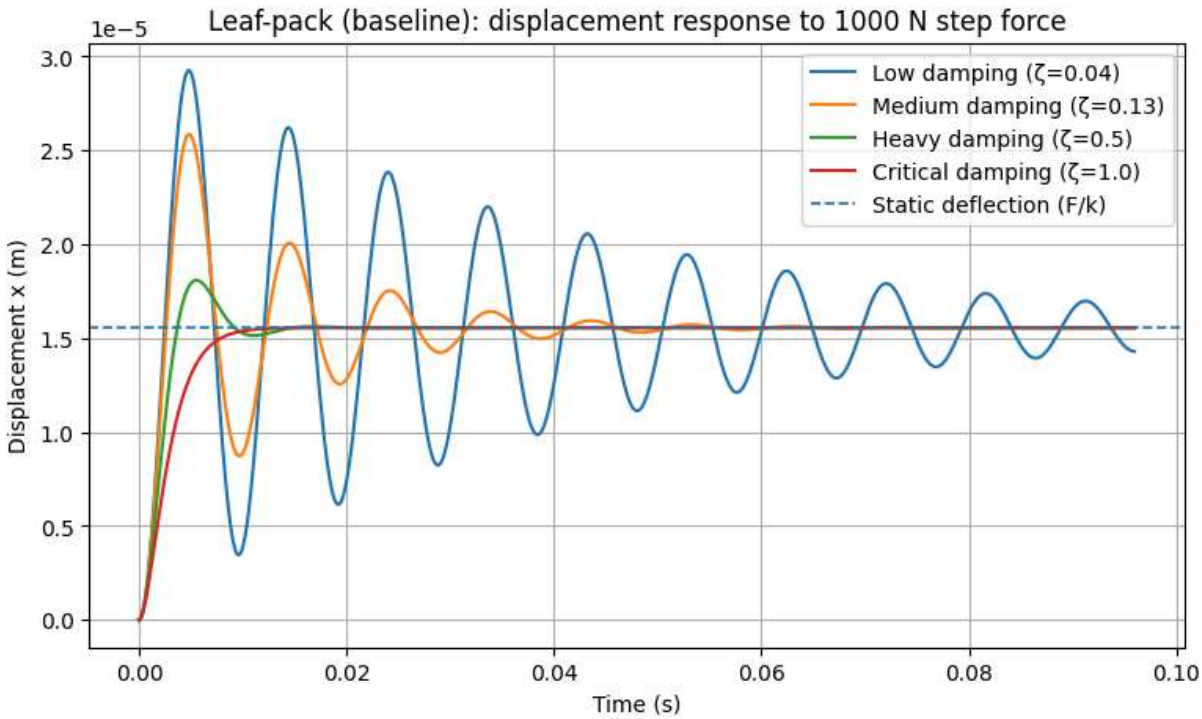
In [65]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
```

In [66]:
```
# k, mass and Force value from Solidworks
k = 6.4309e7      # N/m   how stiff it is
m = 150.0        # kg   mass of lump hanging
F0 = 1000.0      # N    force

# Damping cases (damping ratio zeta)
cases = {
    "Low damping (ζ=0.04)": 0.04,
    "Medium damping (ζ=0.13)": 0.13,
    "Heavy damping (ζ=0.5)": 0.5,
    "Critical damping (ζ=1.0)": 1.0,
}

# calculation
wn = np.sqrt(k / m)              # rad/s
fn = wn / (2*np.pi)              # Hz
Tn = 2*np.pi / wn                # s
x_static = F0 / k                # m
c_crit = 2*np.sqrt(m*k)          # N*s/m

print(f"k = {k:.3e} N/m")
print(f"m = {m:.1f} kg")
print(f"Natural frequency: fn = {fn:.2f} Hz (Tn = {Tn:.6f} s)")
print(f"Static deflection under 1000 N: x = F/k = {x_static:.6e} m ({x_static*1000:
print(f"Critical damping: c_crit = {c_crit:.3e} N·s/m")
```

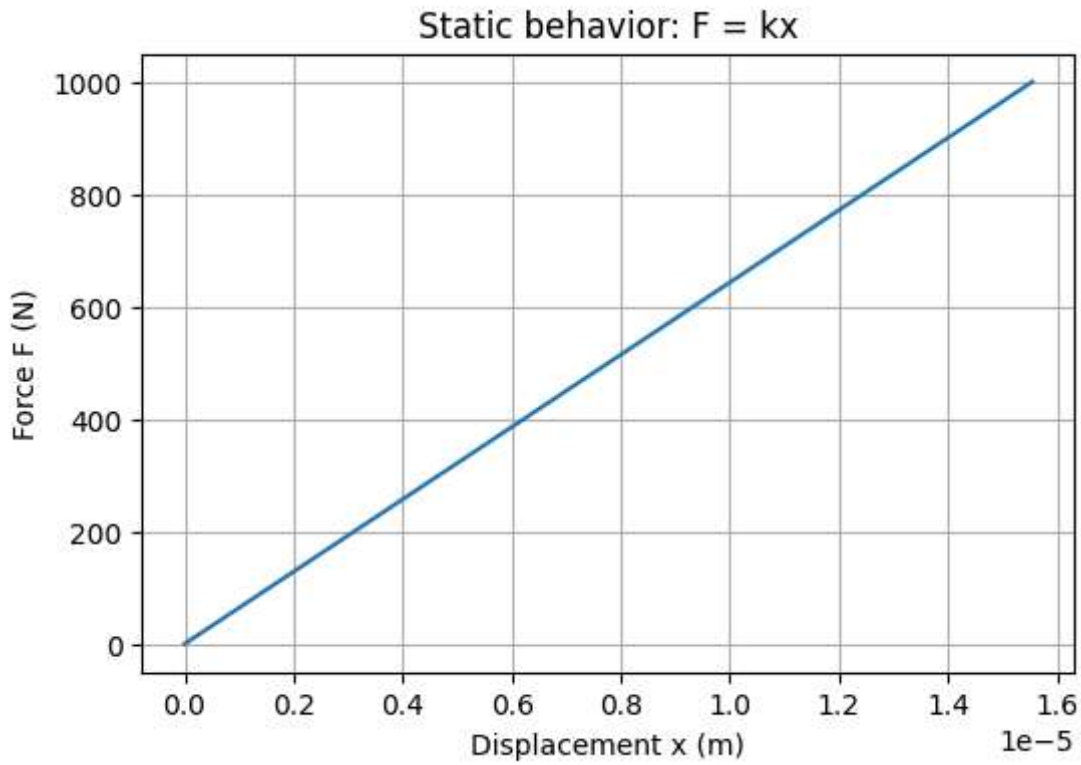
```
k = 6.431e+07 N/m
m = 150.0 kg
Natural frequency: fn = 104.21 Hz (Tn = 0.009596 s)
Static deflection under 1000 N: x = F/k = 1.554992e-05 m (0.015550 mm)
Critical damping: c_crit = 1.964e+05 N·s/m
```

In [67]:
```
def msd_step(t, y, m, c, k, F0):
    """Mass-spring-damper with a step force F0 applied at t>=0."""
    x, v = y
    dxdt = v
    dvdt = (F0 - c*v - k*x) / m
    return [dxdt, dvdt]

# time boundary
t_end = max(0.05, 10*Tn)                # seconds
t_eval = np.linspace(0, t_end, 4000)    # time points
y0 = [0.0, 0.0]                         # start at rest

plt.figure(figsize=(9,5))

for label, zeta in cases.items():
    c = 2*zeta*np.sqrt(m*k)
    sol = solve_ivp(
        msd_step,
```

```
        (0, t_end),
        y0,
        t_eval=t_eval,
        args=(m, c, k, F0),
        rtol=1e-7,
        atol=1e-9
    )
    plt.plot(sol.t, sol.y[0], label=label)

# static deflection
plt.axhline(x_static, linestyle="--", linewidth=1.2, label="Static deflection (F/k)

plt.grid(True)
plt.xlabel("Time (s)")
plt.ylabel("Displacement x (m)")
plt.title("Leaf-pack (baseline): displacement response to 1000 N step force")
plt.legend()
plt.show()
```



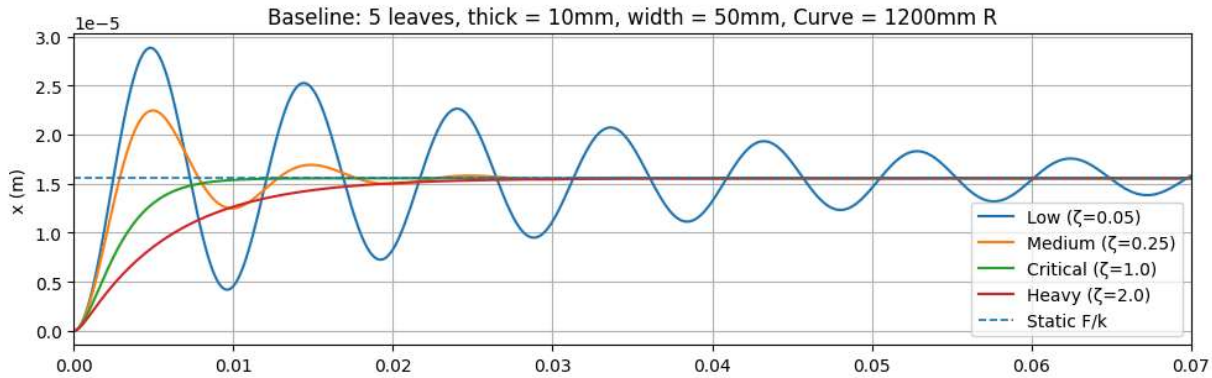
In [68]:
```
F = np.linspace(0, F0, 50)
x = F / k

plt.figure(figsize=(6,4))
plt.plot(x, F)
plt.grid(True)
plt.xlabel("Displacement x (m)")
plt.ylabel("Force F (N)")
plt.title("Static behavior: F = kx")
plt.show()
```

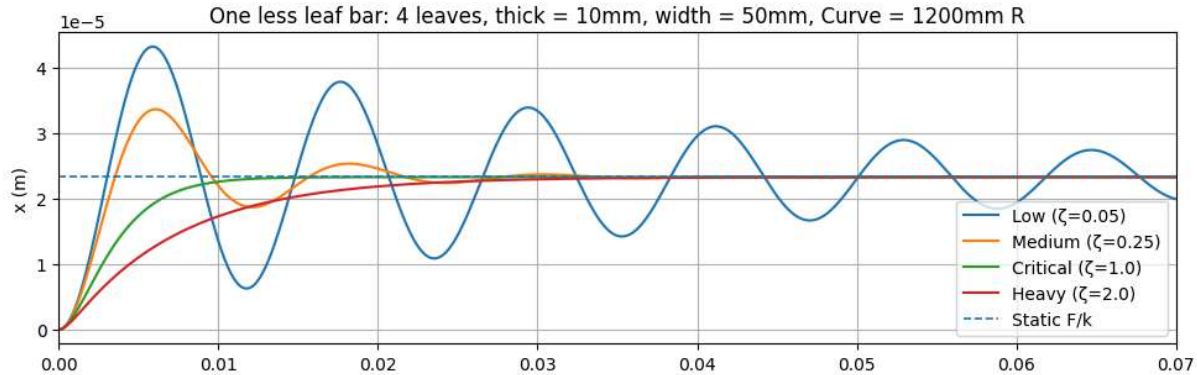


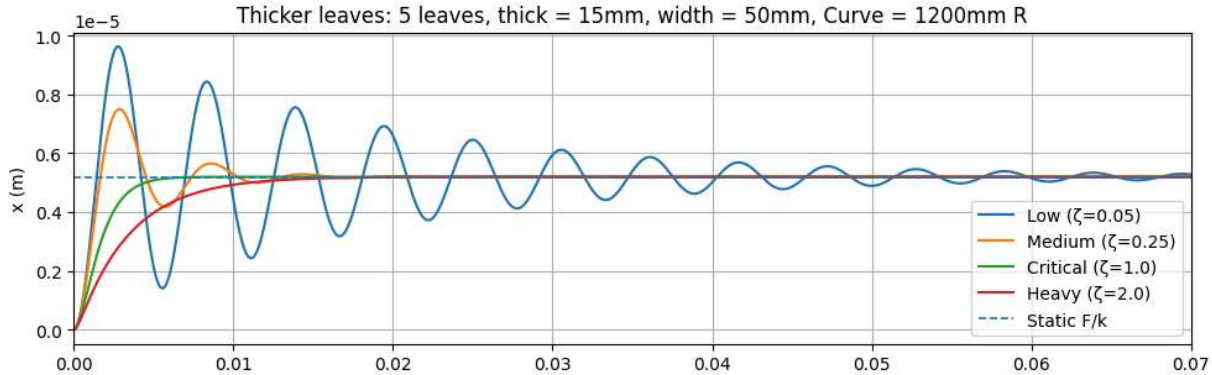
In [69]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Stiffness values (N/m)
k_5_10        = 6.4309e7   # Baseline: 5 leaves, thick = 10mm, width = 50mm, Curve
k_4_10        = 4.2900e7   # One less leaf bar: 4 leaves, thick = 10mm, width = 50m
k_5_15        = 1.9242e8   # Thicker leaves: 5 leaves, thick = 15mm, width = 50mm,
k_5_10_w60    = 7.6746e7   # Wider leaves: 5 leaves, thick = 10mm, width = 60mm, Cu
k_more_curve  = 7.0069e7   # More curved leaves, 5 leaves, thick = 10mm, width = 60

m  = 150.0
F0 = 1000.0

cases = {
    "Low (ζ=0.05)": 0.05,
    "Medium (ζ=0.25)": 0.25,
    "Critical (ζ=1.0)": 1.0,
    "Heavy (ζ=2.0)": 2.0
}

def msd_step(t, y, m, c, k, F0):
    x, v = y
    return [v, (F0 - c*v - k*x)/m]

def simulate(k, zeta):
    wn = np.sqrt(k/m)
    Tn = 2*np.pi/wn
    t_end = max(0.05, 30*Tn)
    t_eval = np.linspace(0, t_end, 4000)

    c = 2*zeta*np.sqrt(m*k)
```

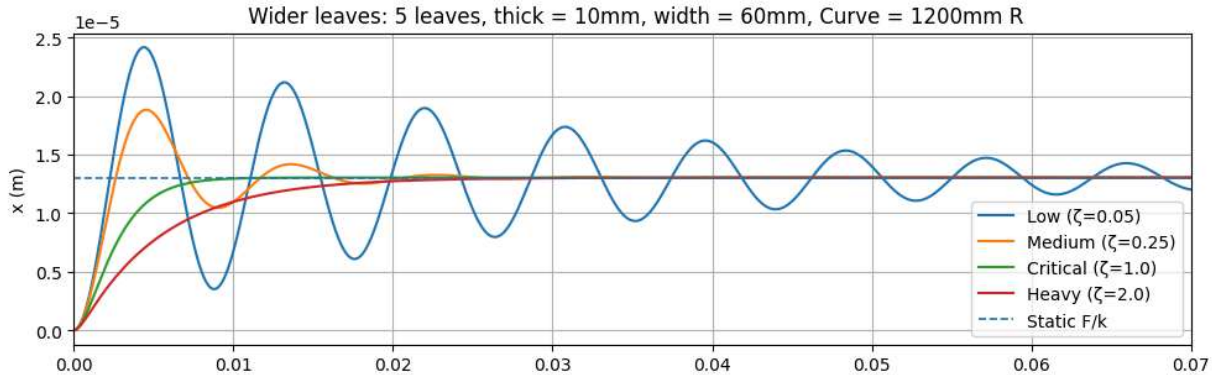
```
    sol = solve_ivp(
        msd_step, (0, t_end), [0, 0],
        t_eval=t_eval, args=(m, c, k, F0),
        rtol=1e-7, atol=1e-9
    )
    return sol.t, sol.y[0]

fig, axes = plt.subplots(5, 1, figsize=(10,16), sharex=False)

# axes[0] : baseline
for label, zeta in cases.items():
    t, x = simulate(k_5_10, zeta)
    axes[0].plot(t, x, label=label)
axes[0].axhline(F0/k_5_10, linestyle="--", linewidth=1.2, label="Static F/k")
axes[0].set_title("Baseline: 5 leaves, thick = 10mm, width = 50mm, Curve = 1200mm R
axes[0].set_ylabel("x (m)")
axes[0].grid(True); axes[0].legend()
axes[0].set_xlim(0, 0.07)

# axes[1] : 4 leaves
for label, zeta in cases.items():
    t, x = simulate(k_4_10, zeta)
    axes[1].plot(t, x, label=label)
axes[1].axhline(F0/k_4_10, linestyle="--", linewidth=1.2, label="Static F/k")
axes[1].set_title("One less leaf bar: 4 leaves, thick = 10mm, width = 50mm, Curve =
axes[1].set_ylabel("x (m)")
axes[1].grid(True); axes[1].legend()
axes[1].set_xlim(0, 0.07)

# axes[2] : thicker 15mm
for label, zeta in cases.items():
    t, x = simulate(k_5_15, zeta)
    axes[2].plot(t, x, label=label)
axes[2].axhline(F0/k_5_15, linestyle="--", linewidth=1.2, label="Static F/k")
axes[2].set_title("Thicker leaves: 5 leaves, thick = 15mm, width = 50mm, Curve = 12
axes[2].set_ylabel("x (m)")
axes[2].grid(True); axes[2].legend()
axes[2].set_xlim(0, 0.07)

# axes[3] : wider 60mm
for label, zeta in cases.items():
    t, x = simulate(k_5_10_w60, zeta)
    axes[3].plot(t, x, label=label)
axes[3].axhline(F0/k_5_10_w60, linestyle="--", linewidth=1.2, label="Static F/k")
axes[3].set_title(" Wider leaves: 5 leaves, thick = 10mm, width = 60mm, Curve = 120
axes[3].set_ylabel("x (m)")
axes[3].grid(True); axes[3].legend()
axes[3].set_xlim(0, 0.07)

#  axes[4] : more curved (R=700mm)
for label, zeta in cases.items():
    t, x = simulate(k_more_curve, zeta)
    axes[4].plot(t, x, label=label)
axes[4].axhline(F0/k_more_curve, linestyle="--", linewidth=1.2, label="Static F/k")
axes[4].set_title("More curved leaves, 5 leaves, thick = 10mm, width = 60mm, Curve
axes[4].set_xlabel("Time (s)")
```

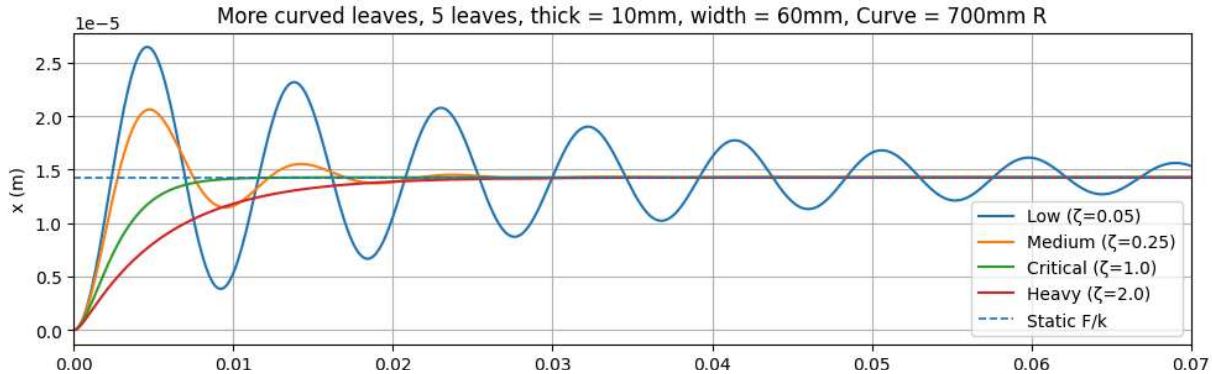
```
axes[4].set_ylabel("x (m)")
axes[4].grid(True); axes[4].legend()
axes[4].set_xlim(0, 0.07)

plt.tight_layout()
plt.show()
```

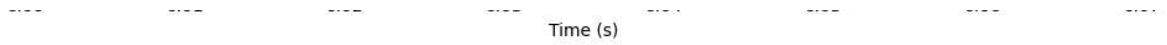

Baseline: 5 leaves, thick = 10mm, width = 50mm, Curve = 1200mm R
1e−5
x (m)
Low (ζ=0.05)
Medium (ζ=0.25)
Critical (ζ=1.0)
Heavy (ζ=2.0)
Static F/k


One less leaf bar: 4 leaves, thick = 10mm, width = 50mm, Curve = 1200mm R
1e−5
x (m)
Low (ζ=0.05)
Medium (ζ=0.25)
Critical (ζ=1.0)
Heavy (ζ=2.0)
Static F/k

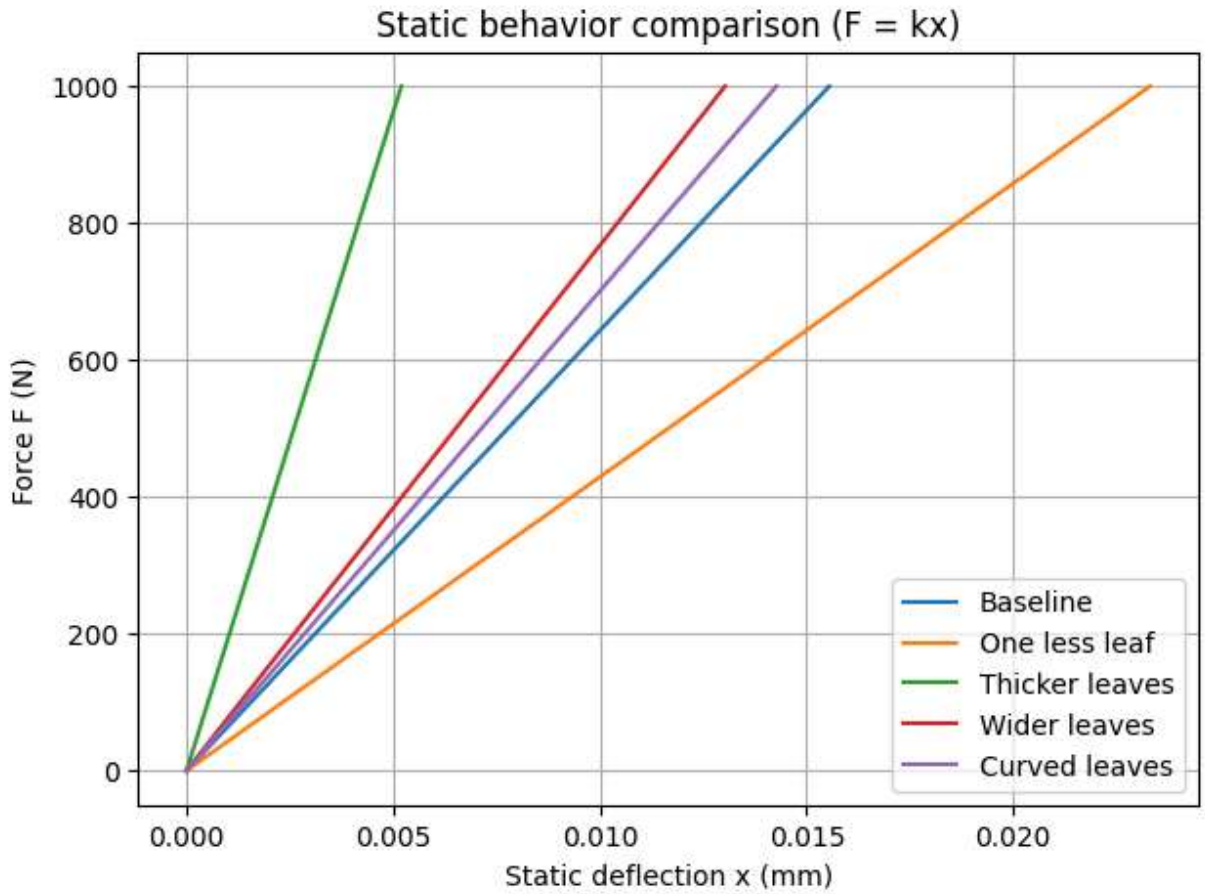

Thicker leaves: 5 leaves, thick = 15mm, width = 50mm, Curve = 1200mm R
1e−5
x (m)
Low (ζ=0.05)
Medium (ζ=0.25)
Critical (ζ=1.0)
Heavy (ζ=2.0)
Static F/k


Wider leaves: 5 leaves, thick = 10mm, width = 60mm, Curve = 1200mm R
1e−5
x (m)
Low (ζ=0.05)
Medium (ζ=0.25)
Critical (ζ=1.0)
Heavy (ζ=2.0)
Static F/k


More curved leaves, 5 leaves, thick = 10mm, width = 60mm, Curve = 700mm R
1e−5
x (m)
Low (ζ=0.05)
Medium (ζ=0.25)
Critical (ζ=1.0)
Heavy (ζ=2.0)
Static F/k



In [70]:
```
import numpy as np
import matplotlib.pyplot as plt

# Stiffness values (N/m)
k_5_10        = 6.4309e7   # baseline
k_4_10        = 4.2900e7   # 4 leaves
k_5_15        = 1.9242e8   # thicker
k_5_10_w60    = 7.6746e7   # wider
k_more_curve  = 7.0069e7   # curve (R=700mm)

Fmax = 1000.0
F = np.linspace(0, Fmax, 200)

def x_mm(F, k):
    return (F / k) * 1000  # m -> mm

plt.figure(figsize=(7,5))
plt.plot(x_mm(F, k_5_10),       F, label="Baseline")
plt.plot(x_mm(F, k_4_10),       F, label="One less leaf")
plt.plot(x_mm(F, k_5_15),       F, label="Thicker leaves")
plt.plot(x_mm(F, k_5_10_w60),   F, label="Wider leaves")
plt.plot(x_mm(F, k_more_curve), F, label="Curved leaves")

plt.grid(True)
plt.xlabel("Static deflection x (mm)")
plt.ylabel("Force F (N)")
plt.title("Static behavior comparison (F = kx)")
plt.legend()
plt.show()

print("Deflection at 1000 N:")
print(f"Baseline (5 leaves, 10mm, b=50): {(Fmax/k_5_10)*1000:.6f} mm")
print(f"4 leaves, 10mm, b=50:            {(Fmax/k_4_10)*1000:.6f} mm")
print(f"5 leaves, 15mm, b=50:            {(Fmax/k_5_15)*1000:.6f} mm")
print(f"5 leaves, 10mm, b=60:            {(Fmax/k_5_10_w60)*1000:.6f} mm")
print(f"More curved (R=700mm):            {(Fmax/k_more_curve)*1000:.6f} mm")
```



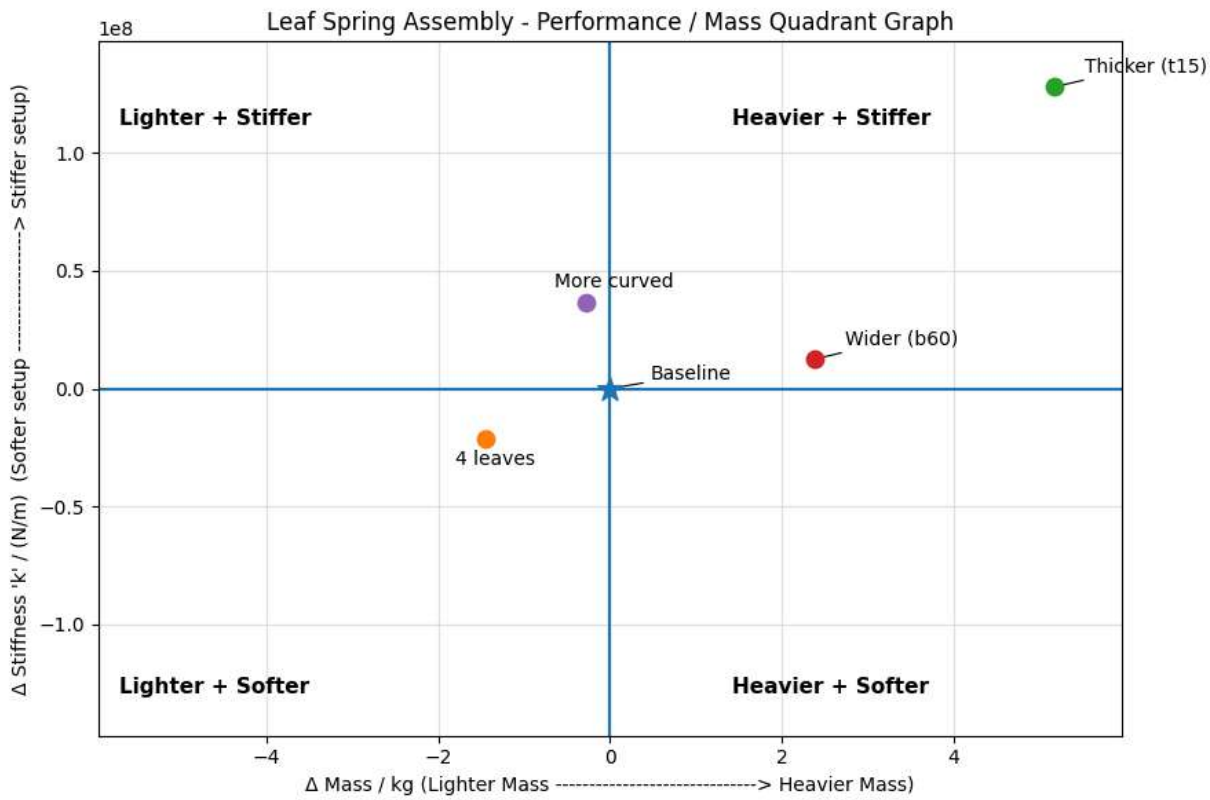

```
Deflection at 1000 N:
Baseline (5 leaves, 10mm, b=50): 0.015550 mm
4 leaves, 10mm, b=50:            0.023310 mm
5 leaves, 15mm, b=50:            0.005197 mm
5 leaves, 10mm, b=60:            0.013030 mm
More curved (R=700mm):            0.014272 mm
```

In [71]:

```python
import numpy as np
import matplotlib.pyplot as plt

# mass in kg, stiffness in N/m
data = {
    "Baseline":      {"m": 13.69, "k": 6.4309e7},
    "4 leaves":      {"m": 12.25, "k": 4.2900e7},
    "Thicker (t15)": {"m": 18.87, "k": 1.9242e8},
    "Wider (b60)":   {"m": 16.07, "k": 7.6746e7},
    "More curved":   {"m": 13.41, "k": 1.0069e8},
}

# Baseline reference
m0 = data["Baseline"]["m"]
k0 = data["Baseline"]["k"]

labels = list(data.keys())
dx = np.array([data[name]["m"] - m0 for name in labels], dtype=float)
dy = np.array([data[name]["k"] - k0 for name in labels], dtype=float)

# Center baseline at 0,0
xmax = np.max(np.abs(dx)) if len(dx) else 1.0
```

```
ymax = np.max(np.abs(dy)) if len(dy) else 1.0
xpad = 0.15 * xmax if xmax > 0 else 1.0
ypad = 0.15 * ymax if ymax > 0 else 1.0

xlim = (-xmax - xpad, xmax + xpad)
ylim = (-ymax - ypad, ymax + ypad)

fig, ax = plt.subplots(figsize=(9,6))

# Crosshair
ax.axvline(0, linewidth=1.6)
ax.axhline(0, linewidth=1.6)

# Points
for name, x, y in zip(labels, dx, dy):
    if name == "Baseline":
        ax.scatter(x, y, s=180, marker="*", zorder=4)
    else:
        ax.scatter(x, y, s=80, zorder=4)

# Label placement
# initial label positions (in data units)
label_xy = []
for name, x, y in zip(labels, dx, dy):
    # Put labels generally away from the origin, and away from the point a bit
    offx = 0.03 * (xlim[1] - xlim[0]) * (1 if x >= 0 else -1)
    offy = 0.03 * (ylim[1] - ylim[0]) * (1 if y >= 0 else -1)
    if name == "Baseline":
        offx, offy = 0.04*(xlim[1]-xlim[0]), 0.02*(ylim[1]-ylim[0])
    label_xy.append([x + offx, y + offy])

label_xy = np.array(label_xy, dtype=float)


# draw labels with leader lines
for name, x, y, (lx, ly) in zip(labels, dx, dy, label_xy):
    ax.annotate(
        name,
        xy=(x, y),
        xytext=(lx, ly),
        textcoords="data",
        fontsize=10,
        arrowprops=dict(arrowstyle="-", lw=0.8),
        va="center"
    )

# Quadrants
ax.text(0.02, 0.88, "Lighter + Stiffer", transform=ax.transAxes, fontsize=11, fontw
ax.text(0.62, 0.88, "Heavier + Stiffer", transform=ax.transAxes, fontsize=11, fontw
ax.text(0.02, 0.06, "Lighter + Softer", transform=ax.transAxes, fontsize=11, fontwe
ax.text(0.62, 0.06, "Heavier + Softer", transform=ax.transAxes, fontsize=11, fontwe

ax.set_xlim(*xlim)
ax.set_ylim(*ylim)

ax.grid(True, alpha=0.35)
```

```
ax.set_xlabel("Δ Mass / kg (Lighter Mass ------------------------------> Heavier Ma
ax.set_ylabel("Δ Stiffness 'k' / (N/m)  (Softer setup -----------------> Stiffer s
ax.set_title("Leaf Spring Assembly - Performance / Mass Quadrant Graph")

plt.tight_layout()
plt.show()
```



In [ ]:

In [ ]: